TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

---------- *** ----------

BÁO CÁO BÀI TẬP 3
KIỂM THỬ VÀ ĐÓNG GÓI PROTOTYPE (20%)

Học phần: Chuyên đề tốt nghiệp 1 (CDTN01) — Học kỳ 1, Năm học 2026 – 2027
Hệ thống: Smart CRM – Phân hệ Mekong Mobile
Luồng nghiệp vụ 05 (L5): Quản lý Kho linh kiện thay thế (Trung tâm Quận 1: L5-WH-890)
Track chuyên ngành: Kỹ thuật phần mềm (Software Engineering – SE)

Bảng TB.1: Thông tin học viên, định danh kho lưu trữ và hồ sơ phát hành chính thức

Họ và tên sinh viên:	Huỳnh Nguyễn Bảo Ngọc
Mã số sinh viên (MSSV):	2374802010339
Lớp học phần:	261_71ITGR40203_06 (Khóa K27 CNTT)
Giảng viên hướng dẫn:	TS. Nguyễn Trí Hải
Link GitHub Repository (Public):	**https://github.com/Johnny120905/cdtn01_HuynhNguyenBaoNgoc_L5.git**
Link Tag v0.1 (Bài tập 1 - Thiết kế):	**https://github.com/Johnny120905/cdtn01_HuynhNguyenBaoNgoc_L5/releases/tag/v0.1**
Link Tag v0.2 (Bài tập 2 - Module cốt lõi):	**https://github.com/Johnny120905/cdtn01_HuynhNguyenBaoNgoc_L5/releases/tag/v0.2**
Link Tag v1.0 & Bản Deploy trực tuyến:	**Tag v1.0: https://github.com/Johnny120905/cdtn01_HuynhNguyenBaoNgoc_L5/releases/tag/v1.0** **Deploy trực tuyến (Render): https://cdtn01.onrender.com**

TP. HỒ CHÍ MINH — THÁNG 11/2026

MỤC LỤC VÀ DANH MỤC TÀI LIỆU BÁO CÁO

Bảng ML.1: Cấu trúc phân đoạn và điều hướng các mục báo cáo theo đề bài BT3

Mục	Nội dung báo cáo chi tiết	Phạm vi trang
MỤC 1	TEST PLAN RÚT GỌN (Phạm vi, Chiến lược 4 tầng, Tiêu chí thành công, Bộ dữ liệu)	Trang 3 – 4
MỤC 2	TEST REPORT — BẢNG 13 TEST CASES CHẠY THẬT ĐỦ 8 CỘT (Biên, Nghiệp vụ, Lũy đẳng, CRUD)	Trang 5 – 6
MỤC 3	BUG LOG — NHẬT KÝ GỠ LỖI (5 Bugs thực tế chuẩn 7 dòng, Phân tích RCA & Commit)	Trang 7 – 8
MỤC 4	DEPLOYMENT & OPERATION GUIDE (Tiếng Anh kỹ thuật, Docker Compose <= 2 lệnh, Cloud)	Trang 9 – 10
MỤC 5	CHECKLIST BẢO MẬT 7 MỤC ĐÃ TỰ ĐÁNH GIÁ (OWASP, RBAC, Bcrypt, NoSQLi, XSS)	Trang 11
MỤC 6	TÀI LIỆU KỸ THUẬT THEO TRACK SE (Clean Architecture, RESTful API 12 endpoints, DB)	Trang 12 – 14
MỤC 7	ẢNH CHỤP MINH CHỨNG VÀ KẾT QUẢ PHÉP THỬ MÁY TRẮNG (Test PASS, Docker, 3 UI States)	Trang 15 – 18
PHỤ LỤC	BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ TRÍ TUỆ NHÂN TẠO (AI) & CAM KẾT LIÊM CHÍNH	Trang 19

Danh mục Bảng biểu và Danh mục Hình ảnh trong báo cáo:

• Bảng TB.1: Thông tin học viên, định danh kho lưu trữ và hồ sơ phát hành chính thức (Trang 1)
• Bảng ML.1: Cấu trúc phân đoạn và điều hướng các mục báo cáo theo đề bài BT3 (Trang 2)
• Bảng 1.1: Bảng ma trận phân loại tầng kiểm thử và mục tiêu chất lượng hệ thống (Trang 3)
• Bảng 2.1: Bảng tổng hợp kết quả thực thi 13 ca kiểm thử trọng yếu (Test Execution Report) (Trang 5)
• Bảng 3.1: Bảng tổng hợp vòng đời và phân loại 5 lỗi phần mềm (Bug Lifecycle Matrix) (Trang 7)
• Bảng 4.1: Danh mục thông số cấu hình và biến môi trường hệ thống (.env) (Trang 9)
• Bảng 5.1: Bảng tự đánh giá Checklist bảo mật 7 tiêu chí OWASP và Chuẩn Khoa CNTT (Trang 11)
• Bảng 6.1: Danh mục đặc tả RESTful API Endpoints cho hệ thống Quản lý Kho linh kiện (Trang 13)
• Bảng 6.2: Từ điển dữ liệu (Data Dictionary) các thuộc tính thực thể cơ sở dữ liệu MongoDB (Trang 14)
• Bảng PL.1: Bảng khai báo minh bạch việc sử dụng công cụ AI trong Bài tập 3 (Trang 21)
• Hình 6.1: Sơ đồ kiến trúc phân tầng sạch (Clean Architecture) và luồng dữ liệu nghiệp vụ (Trang 13)
• Hình 7.1: Phép thử máy trắng Bước 1 — Tạo thư mục D:\kiem-tra-may-trang và Clone mã nguồn Tag v1.0 (Trang 15)
• Hình 7.2: Phép thử máy trắng Bước 2 — Khởi dựng Docker Compose Up thành công 2/2 Container (Trang 15)
• Hình 7.3: Phép thử máy trắng Bước 3 — Trạng thái Container Up (healthy) & API Health Check (Trang 16)
• Hình 7.4: Phép thử máy trắng Bước 4a — Thực thi kiểm thử tự động toàn diện (Phần 1: 1% đến 49%) (Trang 16)
• Hình 7.5: Phép thử máy trắng Bước 4b — Hoàn thành 100% kiểm thử 67/67 Test Cases PASSED (Trang 16)
• Hình 7.6: Minh chứng Kiểm chứng BUG-01 — Mật khẩu mã hóa Bcrypt một chiều (test_09 PASSED) (Trang 17)
• Hình 7.7: Minh chứng Kiểm chứng BUG-02 — Thời hạn mã OTP quên mật khẩu có giới hạn 10 phút (test_06 PASSED) (Trang 17)
• Hình 7.8: Minh chứng Kiểm chứng BUG-03 — Nạp dữ liệu hạt giống Seed Data lũy đẳng (seeds.seed) (Trang 17)
• Hình 7.9: Minh chứng Kiểm chứng BUG-04 — Luồng cảnh báo thiếu hụt kho kích hoạt Quản lý (test_01) (Trang 18)
• Hình 7.10: Minh chứng Kiểm chứng BUG-05 — Đầy đủ 11 bài test tự động cho thao tác XÓA an toàn (Trang 18)
• Hình 7.11: Giao diện Web Prototype — Trạng thái Đang tải dữ liệu (Loading State với Shimmer) (Trang 19)
• Hình 7.12: Giao diện Web Prototype — Trạng thái Rỗng (Empty State khi kho an toàn) (Trang 19)
• Hình 7.13: Giao diện Web Prototype — Trạng thái Lỗi nghiệp vụ (Error State cảnh báo vi phạm tồn) (Trang 20)
• Hình 7.14: Bảng điều khiển Quản lý kho (Manager Inventory Dashboard & Alert Metrics) (Trang 20)
• Hình 7.15: Giao diện lập phiếu nhập kho linh kiện dành cho Nhân viên Vật tư (Material Staff) (Trang 20)
• Hình 7.16: Giao diện tra cứu và lập phiếu xuất kho gắn mã bảo hành dành cho Kỹ thuật viên (Trang 20)

MỤC 1: TEST PLAN RÚT GỌN

Mục tiêu kiểm thử của Bài tập 3 tập trung vào việc xác nhận tính toàn vẹn của logic nghiệp vụ kho, kiểm định khả năng phòng thủ dữ liệu biên, chứng minh cơ chế bảo mật xác thực phân quyền RBAC và đảm bảo hệ thống có thể đóng gói, khởi dựng độc lập bằng đúng <= 2 lệnh trên bất kỳ máy tính trắng nào.

1.1. Phạm vi kiểm thử (In-Scope) và Phần chủ ý KHÔNG kiểm (Out-of-Scope)

• **Phạm vi ĐƯỢC kiểm thử (In-Scope):** Toàn bộ 5 quy tắc nghiệp vụ kho cốt lõi được đặc tả trong SRS bao gồm: BR-01 (Chống xuất kho âm, kiểm soát tồn khả dụng thời gian thực), BR-02 (Bắt buộc mã phiếu bảo hành hợp lệ và đang mở khi xuất hoặc hoàn linh kiện), BR-03 (Sổ cái giao dịch bất biến insert-only, ghi nhận đầy đủ nhân sự và thời gian), BR-04 (Cách ly dữ liệu chi nhánh nghiêm ngặt theo chuẩn OWASP A01: Broken Access Control), BR-05 (Tự động đánh giá và kích hoạt cờ cảnh báo ngưỡng an toàn WARNING/NORMAL). Ngoài ra, phạm vi kiểm thử bao hàm cơ chế kiểm soát xung đột tương tranh Optimistic Concurrency Control qua trường version (HTTP 409 Conflict), toàn bộ quy trình xác thực JWT 24h, mã hóa Bcrypt mật khẩu, chu trình OTP quên mật khẩu có thời hạn 10 phút, kiểm tra tiêu chí SE S1 với đầy đủ 4 thao tác CRUD cho 2 thực thể part và users kèm ràng buộc an toàn (chặn xóa linh kiện còn tồn kho > 0 và chặn người dùng tự xóa chính mình), cùng việc xử lý trọn vẹn 3 trạng thái giao diện UI (Loading, Empty, Error).

• **Phần chủ ý KHÔNG kiểm thử (Out-of-Scope) & Lập luận kỹ thuật:** 1) Gửi SMS/Email viễn thông thực tế (Hệ thống sử dụng bộ mô phỏng máy phát sinh OTP tự động trong bộ nhớ để đảm bảo tính độc lập self-contained trên máy tính trắng khi chấm điểm mà không đòi hỏi tài khoản Gateway trả phí). 2) Thanh toán tài chính và xuất hóa đơn VAT (Nghiệp vụ thuộc Luồng 08 - Kế toán, nằm ngoài biên giới phân hệ Luồng 05). 3) Kiểm thử tải cực hạn (Stress Testing) trên hàng triệu giao dịch đồng thời (Quy mô đồ án CDTN1 tập trung vào tính đúng đắn logic của prototype; tải quy mô lớn sẽ thực hiện trong Chuyên đề 2).

1.2. Chiến lược kiểm thử tự động đa tầng (Testing Pyramid) và Ma trận phân loại

Hệ thống áp dụng mô hình Kim tự tháp kiểm thử phần mềm tiêu chuẩn với tổng cộng 67 ca kiểm thử tự động, được thực thi hoàn toàn thông qua framework pytest 9.2.0. Ma trận phân bổ kiểm thử được tổng hợp trong Bảng 1.1 dưới đây:

Bảng 1.1: Bảng ma trận phân loại tầng kiểm thử và mục tiêu chất lượng hệ thống

Tầng kiểm thử	Tệp kiểm thử phụ trách	Số lượng	Trọng tâm kiểm định chất lượng
Tầng 1: Unit Tests (Kiểm thử đơn vị)	tests/inventory.service.test.py tests/bt3_specification_and_constraints.test.py	20 tests	Logic tính toán tồn kho, kiểm tra giá trị biên hai phía, Optimistic Locking, băm mật khẩu Bcrypt, tính lũy đẳng nạp seed.
Tầng 2: Integration Tests (Tích hợp)	tests/inventory.routes.test.py tests/se_s1_crud.test.py	17 tests	Hợp đồng API RESTful, mã trạng thái HTTP, tính tương thích dữ liệu JSON, trọn vẹn 4 thao tác CRUD cho part và users.
Tầng 3: Security & RBAC (An ninh)	tests/auth_and_user.test.py	15 tests	Xác thực JWT, phân quyền 3 vai trò, độ phức tạp mật khẩu 4 nhóm ký tự, kiểm soát OTP 10 phút, chặn tài khoản vô hiệu hóa TERMINATED.
Tầng 4: Cross-Role & UI/UX (Liên thông)	tests/cross_role_and_ui.test.py	15 tests	Luồng cộng tác giữa Kỹ thuật viên - Nhân viên Vật tư - Quản lý kho, đồng bộ dữ liệu thời gian thực và 3 trạng thái giao diện UI.

1.3. Tiêu chí thành công ĐO LƯỜNG ĐƯỢC (Quantitative Acceptance Criteria)

1) Tỷ lệ vượt qua kiểm thử tự động đạt 100% (67/67 ca kiểm thử đều đạt PASSED, không có test nào bị skipped hay failed). 2) Số lượng lỗi mở mức độ Nghiêm trọng (Critical/Major): 0 lỗi. 3) Tỷ lệ lỗi máy chủ HTTP 500 hoặc lộ vết ngăn xếp hệ thống (Stack Trace): 0.0% (mọi ngoại lệ đều được bọc xử lý trả về mã lỗi 4xx rõ ràng). 4) Thời gian thực thi toàn bộ suite test: < 2.0 giây trên Docker. 5) Số lệnh khởi dựng hệ thống trên máy trắng: Đúng <= 2 lệnh (Copy-Item .env.example .env và docker compose up -d --build).

• **Cơ sở lý luận về tính đầy đủ của kiểm thử:** Luận giải câu hỏi phỏng vấn then chốt: 'Làm sao em biết là em đã kiểm thử ĐỦ?':
Em khẳng định bộ kiểm thử của mình đạt độ bao phủ toàn diện vì được thiết kế dựa trên kỹ thuật Phân tích giá trị biên (Boundary Value Analysis - BVA) và Phân vùng tương đương (Equivalence Partitioning). Cụ thể đối với 5 quy tắc nghiệp vụ BR-01..BR-05, mỗi quy tắc đều có ít nhất 1 test luồng hợp lệ và 2 test biên đối nghịch (ví dụ: xuất kho đúng bằng số lượng tồn kho khả dụng thì cho phép và chuyển trạng thái về 0; nhưng xuất kho vượt quá 1 đơn vị thì lập tức bị chặn với lỗi HTTP 400 ERR_NEGATIVE_STOCK mà không làm sai lệch tồn kho). Hệ thống đồng thời có 2 test dữ liệu sai định dạng để chứng minh không bị sập nguồn 500; 1 test kiểm thử an ninh chi nhánh A01 OWASP; 1 test chứng minh tính lũy đẳng (Idempotence) nạp dữ liệu hạt giống Seed C8; và 11 test cho tiêu chí SE S1 kiểm tra đầy đủ 4 thao tác CRUD (có thao tác xóa DELETE an toàn) cho cả danh mục linh kiện và nhân sự. Toàn bộ 67 ca kiểm thử đều chạy tự động và PASS 100%.

1.4. Môi trường kiểm thử và Cơ chế cô lập dọn dẹp dữ liệu (Test Isolation)

Bộ kiểm thử được thiết lập để thực thi độc lập trên môi trường Docker (python:3.11-slim, mongo:7.0) hoặc môi trường Python ảo (.venv). Bộ dữ liệu kiểm thử được định nghĩa trong seeds/001_seed_parts_demo.json bao gồm 40 linh kiện mẫu chuẩn hóa, 75 giao dịch tồn kho tại Trung tâm Quận 1 (L5-WH-890) và Quận 3 (L5-WH-891). Nhằm ngăn ngừa ô nhiễm dữ liệu chéo giữa các bài test (Cross-test Pollution), mỗi test class đều triển khai phương thức setUp và tearDown cô lập: các bài test đơn vị sử dụng kho lưu trữ bộ nhớ RAM (_memory_store) và mock repository; các bài test tích hợp dọn dẹp sạch các collection thử nghiệm ngay sau khi hoàn tất, đảm bảo khả năng thực thi lặp lại vô hạn lần mà kết quả luôn nhất quán 100%.

MỤC 2: TEST REPORT — BẢNG 13 TEST CASES CHẠY THẬT ĐỦ 8 CỘT

Dưới đây là Bảng tổng hợp chi tiết 13 ca kiểm thử trọng yếu tiêu biểu nhất đại diện cho toàn bộ 67 ca kiểm thử tự động của hệ thống, tuân thủ đầy đủ cấu trúc 8 cột quy chuẩn theo đề bài (Mã test, Tiền điều kiện, Các bước thực hiện, Dữ liệu đầu vào, Kết quả mong đợi, Mức độ ưu tiên, Kết quả thực tế, Trạng thái):

Bảng 2.1: Bảng tổng hợp kết quả thực thi 13 ca kiểm thử trọng yếu (Test Execution Report)

Mã	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Mức	Kết quả thực tế	Trạng thái
TC-01 (BR-01)	SCR-IP14-01 tồn = 10 tại L5-WH-890, phiếu TK-1092 mở.	1. Đăng nhập Kỹ thuật. 2. POST /transactions xuất kho 3 cái.	part_code: SCR-IP14-01 type: OUT, qty: 3 ticket_id: TK-1092	HTTP 201 Created, tồn kho giảm từ 10 xuống 7, sinh mã giao dịch.	**CRIT**	HTTP 201 Created, tồn kho giảm còn đúng 7 cái.	**PASS**
TC-02 (BR-02)	BAT-IP13-01 có tồn = 8, phiếu bảo hành TK-1093 mở.	1. Đăng nhập Kỹ thuật. 2. Xuất kho gắn mã phiếu bảo hành.	part_code: BAT-IP13-01 type: OUT, qty: 1 ticket_id: TK-1093	HTTP 201 Created, sổ cái lưu vết gắn chặt mã phiếu bảo hành.	**CRIT**	HTTP 201 Created, trường ticket_id khớp chuẩn TK-1093.	**PASS**
TC-03 (FR-03)	Kỹ thuật viên có linh kiện dư sau khi sửa chữa.	1. Đăng nhập Kỹ thuật. 2. POST /transactions loại RETURN.	part_code: SCR-IP14-01 type: RETURN, qty: 2 ticket_id: TK-1092	HTTP 201 Created, tồn kho tăng từ 10 lên 12, sổ cái ghi loại RETURN.	**MAJOR**	HTTP 201 Created, tồn kho tăng chính xác lên 12 cái.	**PASS**
TC-04 (Biên 1)	Linh kiện có tồn khả dụng đúng 10 cái tại kho.	1. Gửi request xuất kho đúng bằng 10 cái.	part_code: SCR-IP14-01 qty: 10 ticket_id: TK-1092	HTTP 201 Created, tồn kho giảm về đúng 0, cờ trạng thái chuyển WARNING.	**HIGH**	HTTP 201 Created, tồn kho = 0, cờ trạng thái WARNING.	**PASS**
TC-05 (Biên 2)	Linh kiện có tồn khả dụng đúng 10 cái tại kho.	1. Gửi request xuất kho số lượng 11 cái.	part_code: SCR-IP14-01 qty: 11 ticket_id: TK-1092	HTTP 400 Bad Request, ném ERR_NEGATIVE_STOCK, tồn kho giữ nguyên 10.	**CRIT**	HTTP 400 Bad Request, báo lỗi xuất quá tồn, tồn không đổi.	**PASS**
TC-06 (Biên 3)	Ngưỡng an toàn tối thiểu min_threshold = 15.	1. Cập nhật tồn về 15. 2. GET /stock/SCR-IP14-01.	center_id: L5-WH-890 part_code: SCR-IP14-01 quantity: 15	HTTP 200 OK, trường status_flag = WARNING kích hoạt cảnh báo thiếu hụt.	**HIGH**	HTTP 200 OK, status_flag = WARNING, kích hoạt cảnh báo.	**PASS**
TC-07 (Biên 4)	Ngưỡng an toàn tối thiểu min_threshold = 15.	1. Cập nhật tồn lên 16. 2. GET /stock/SCR-IP14-01.	center_id: L5-WH-890 part_code: SCR-IP14-01 quantity: 16	HTTP 200 OK, status_flag = NORMAL, không phát sinh cảnh báo thiếu.	**HIGH**	HTTP 200 OK, status_flag = NORMAL, kho an toàn.	**PASS**

Mã	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Mức	Kết quả thực tế	Trạng thái
TC-08 (Sai 1)	Phiếu bảo hành TK-COMPLETED-01 đã đóng.	1. Kỹ thuật viên cố tình dùng phiếu đã đóng để xuất.	type: OUT, qty: 1 ticket_id: TK-COMPLETED-01	HTTP 422 Unprocessable Entity, báo lỗi phiếu đã đóng hoặc không hợp lệ.	**HIGH**	HTTP 422 ERR_INVALID_TICKET, từ chối xuất kho thành công.	**PASS**
TC-09 (Sai 2)	Người dùng nhập số lượng giao dịch là 0 hoặc âm.	1. POST /transactions với quantity: 0 hoặc -5.	part_code: SCR-IP14-01 type: IN, quantity: 0	HTTP 400 Bad Request, ném ERR_INVALID_QUANTITY, hệ thống không sập.	**HIGH**	HTTP 400 Bad Request, thông báo lỗi số lượng không hợp lệ.	**PASS**
TC-10 (OWASP)	Token thuộc tài khoản Chi nhánh Quận 3 (L5-WH-891).	1. Gửi request truy vấn dữ liệu kho Quận 1.	Bearer Token: L5-WH-891 param: L5-WH-890	HTTP 403 Forbidden, ném ERR_UNAUTHORIZED_CENTER, chặn truy cập chéo.	**CRIT**	HTTP 403 Forbidden, bảo vệ cách ly dữ liệu chi nhánh 100%.	**PASS**
TC-11 (SES3)	Đăng ký người dùng mới hoặc đặt lại mật khẩu.	1. Đăng ký tài khoản mật khẩu thô Password123@. 2. Kiểm tra CSDL.	email: sec.tech@gmail.com pwd: Password123@	CSDL MongoDB chỉ lưu chuỗi băm Bcrypt, tuyệt đối không lưu thô.	**CRIT**	CSDL chỉ lưu password_hash Bcrypt, không có trường thô.	**PASS**
TC-12 (C8 Seed)	CSDL đã nạp dữ liệu mẫu ban đầu.	1. Chạy seed.py lần 1. 2. Chạy lại seed.py lần 2.	Tệp seed 40 linh kiện và 75 tồn kho	Lần 2 thực thi thành công không lỗi, số lượng linh kiện giữ đúng 40.	**CRIT**	Lần 2 hoàn tất 100%, 0 lỗi, 0 nhân đôi dữ liệu (Idempotent).	**PASS**
TC-13 (SES1)	Quản lý kho quản trị danh mục và nhân sự.	1. Tạo mới -> 2. Đọc -> 3. Sửa -> 4. Thử xóa lỗi -> 5. Xóa an toàn.	POST/GET/PUT/DELETE trên /parts và /users	Đầy đủ 4 thao tác CRUD, chặn xóa khi tồn > 0 và chặn tự xóa chính mình.	**CRIT**	Hoàn thành trọn vẹn CRUD 2 thực thể, ràng buộc an toàn 100%.	**PASS**

Phân tích và Đánh giá Kết quả Thực thi Kiểm thử tự động:

Toàn bộ 67 ca kiểm thử tự động của hệ thống được thực thi trong thời gian 1.13 giây trên container Docker (hoặc 1.25 giây trên môi trường máy trạm cục bộ .venv). Kết quả đạt tỷ lệ thành công 100% (67 passed, 0 failed, 0 error). Tất cả các trường hợp ngoại lệ từ chối xuất kho (tồn kho âm, phiếu bảo hành không hợp lệ, dữ liệu sai định dạng) đều trả về đúng mã trạng thái HTTP quy chuẩn (400 Bad Request, 403 Forbidden, 422 Unprocessable Entity) kèm thông điệp giải thích nghiệp vụ bằng tiếng Việt rõ ràng, ngăn chặn hoàn toàn nguy cơ sập máy chủ HTTP 500 hay rò rỉ vết ngăn xếp mã nguồn ra ngoài.

MỤC 3: BUG LOG — NHẬT KÝ THEO DÕI VÀ GỠ LỖI

Trong quá trình kiểm thử hệ thống và rà soát theo các nhận xét góp ý từ Bài tập 2 của Giảng viên hướng dẫn, nhóm phát triển đã ghi nhận, phân loại và khắc phục triệt để 5 lỗi phần mềm thực tế. Dưới đây là Bảng tổng hợp vòng đời lỗi và chi tiết từng lỗi đúng quy chuẩn 7 dòng:

Bảng 3.1: Bảng tổng hợp vòng đời và phân loại 5 lỗi phần mềm (Bug Lifecycle Matrix)

Mã Bug	Tên lỗi phần mềm	Mức độ	Trạng thái	Commit Hash minh chứng
BUG-01	Lỗ hổng lưu mật khẩu Plaintext dạng rõ trong CSDL và RAM	CRITICAL	**CLOSED / VERIFIED**	Commit 23a7755 (và bbb5232)
BUG-02	Thời hạn OTP quên mật khẩu vô hạn, không có cơ chế hết hạn	HIGH	**CLOSED / VERIFIED**	Commit 23a7755 (và bbb5232)
BUG-03	Lỗi xung đột trùng lặp khóa chính khi nạp dữ liệu mẫu seed.py	CRITICAL	**CLOSED / VERIFIED**	Commit 23f1563 (và bbb5232)
BUG-04	Luồng cảnh báo thiếu hụt kho không phản hồi vai trò Quản lý	MEDIUM	**CLOSED / VERIFIED**	Commit 23f1563
BUG-05	Thiếu 11 bài test thao tác XÓA an toàn (Nhận xét BT2 của Thầy)	HIGH	**CLOSED / VERIFIED**	Commit 04a015b

Chi tiết 5 Lỗi Phần Mềm được Khắc Phục Triệt Để (Mỗi Bug đủ 7 dòng quy chuẩn):

BUG-01: Lỗ hổng lưu mật khẩu dạng rõ trong CSDL và RAM (Vi phạm tiêu chuẩn an ninh SE S3)
1. Tóm tắt lỗi: Khi người dùng đăng ký hoặc đổi mật khẩu, CSDL MongoDB và file JSON lưu trữ cả trường password thô.
2. Môi trường phát hiện: Python 3.11, FastAPI, MongoDB 7.0 trên máy trạm phát triển cục bộ và Docker.
3. Các bước tái hiện: 1) Gửi POST /api/v1/auth/register với email và password thô; 2) Mở CSDL kiểm tra collection users.
4. Kết quả thực tế: Bản ghi CSDL xuất hiện trường password: 'Password123@' dạng văn bản rõ không mã hóa.
5. Kết quả mong đợi: CSDL tuyệt đối không được lưu password thô; chỉ được lưu password_hash đã băm an toàn.
6. Nguyên nhân gốc rễ: Trong repository lớp lưu trữ, đối tượng user_doc gán trực tiếp raw_pwd để kiểm tra nhanh mà quên loại bỏ trước khi lưu.
7. Cách khắc phục & Commit: Loại bỏ triệt để việc gán trường password thô, áp dụng hàm băm một chiều Bcrypt với Salt 12 rounds. Minh chứng bài test test_09 trong bt3_specification_and_constraints.test.py PASS 100% tại Commit 23a7755.

BUG-02: Thời hạn mã OTP quên mật khẩu không có giới hạn hết hạn (Lỗ hổng bảo mật Brute-force)
1. Tóm tắt lỗi: Mã OTP cấp lại mật khẩu có giá trị vĩnh viễn, người dùng có thể dùng mã sinh từ nhiều ngày trước để đặt lại mật khẩu.
2. Môi trường phát hiện: Module Xác thực (Auth Service) trên môi trường kiểm thử Docker.
3. Các bước tái hiện: 1) Gọi POST /api/v1/auth/forgot-password; 2) Chờ 15 phút; 3) Gửi POST /api/v1/auth/reset-password với OTP cũ.
4. Kết quả thực tế: Hệ thống chấp nhận mã OTP cũ và cho phép đặt lại mật khẩu mới thành công.
5. Kết quả mong đợi: Mã OTP phải hết hiệu lực sau đúng 10 phút; nếu quá hạn phải trả về lỗi HTTP 400 OTP_EXPIRED.
6. Nguyên nhân gốc rễ: Dictionary _otp_store chỉ lưu cặp {email: otp_code} mà không lưu trường mốc thời gian expires_at.
7. Cách khắc phục & Commit: Cấu trúc lại _otp_store lưu {'code': otp, 'expires_at': now + 600s}; kiểm tra datetime.utcnow() > expires_at. Minh chứng bài test test_06 trong auth_and_user.test.py PASS 100% tại Commit 23a7755.

BUG-03: Lỗi xung đột trùng lặp khóa chính khi thực thi tệp nạp dữ liệu mẫu seeds/seed.py lần 2 (Vi phạm C8)
1. Tóm tắt lỗi: Chạy lệnh nạp dữ liệu seed.py lần thứ hai gây ra lỗi DuplicateKeyError do cố gắng chèn lại part_code đã tồn tại.
2. Môi trường phát hiện: MongoDB 7.0, kịch bản triển khai tự động hóa bằng Docker Compose.
3. Các bước tái hiện: 1) Khởi động container và chạy python -m seeds.seed lần 1; 2) Chạy lại python -m seeds.seed lần 2.
4. Kết quả thực tế: Quá trình seed bị ngắt quãng giữa chừng với ngoại lệ pymongo.errors.DuplicateKeyError.
5. Kết quả mong đợi: Kịch bản nạp seed phải đảm bảo tính lũy đẳng (Idempotent): chạy nhiều lần vẫn thành công và giữ nguyên 40 linh kiện.
6. Nguyên nhân gốc rễ: Mã nguồn sử dụng phương thức insert_many() không điều kiện thay vì cơ chế cập nhật upsert.
7. Cách khắc phục & Commit: Chuyển đổi toàn bộ logic seed sang bulk_write với update_one(filter, {'$set': doc}, upsert=True). Minh chứng bài test test_12 trong bt3_specification_and_constraints.test.py PASS 100% tại Commit 23f1563.

BUG-04: Luồng cảnh báo tồn kho thiếu phản hồi trực quan trên giao diện Quản lý (Vi phạm FR-06)
1. Tóm tắt lỗi: Khi linh kiện giảm xuống dưới ngưỡng an toàn, cờ WARNING có bật trong API nhưng giao diện Quản lý không làm nổi bật.
2. Môi trường phát hiện: Trình duyệt web Chrome/Edge trên giao diện SPA Frontend.
3. Các bước tái hiện: 1) Xuất kho đưa tồn về dưới ngưỡng; 2) Đăng nhập tài khoản Quản lý; 3) Quan sát danh sách linh kiện.
4. Kết quả thực tế: Dòng linh kiện thiếu hụt hiển thị màu sắc bình thường, không có huy hiệu đỏ cảnh báo và thiếu bộ lọc nhanh.
5. Kết quả mong đợi: Các mặt hàng thiếu hụt phải được làm nổi bật màu đỏ, có badge WARNING và hiển thị tổng số cảnh báo trên Dashboard.
6. Nguyên nhân gốc rễ: Hàm render bảng linh kiện tại app.js chỉ kiểm tra trường quantity > 0 mà quên so sánh với min_threshold.
7. Cách khắc phục & Commit: Bổ sung logic kiểm tra is_warning = item.quantity <= item.min_threshold; tô màu cảnh báo đỏ và gắn huy hiệu. Minh chứng bài test test_01 trong cross_role_and_ui.test.py PASS 100% tại Commit 23f1563.

BUG-05: Thiếu 11 bài test tự động cho thao tác XÓA an toàn (Khắc phục nhận xét Bài tập 2 của Thầy)
1. Tóm tắt lỗi: Báo cáo BT2 nhận xét thiếu các ca kiểm thử cho thao tác XÓA (DELETE) trong bộ tiêu chí SE S1 CRUD.
2. Môi trường phát hiện: Bộ kiểm thử tự động của Bài tập 2.
3. Các bước tái hiện: 1) Mở tệp kiểm thử tests/; 2) Kiểm tra các bài test có chứa phương thức client.delete().
4. Kết quả thực tế: Chỉ có test cho Create, Read, Update; chưa có test chuyên biệt kiểm tra việc xóa linh kiện và người dùng.
5. Kết quả mong đợi: Phải có đầy đủ bài test kiểm tra việc xóa thành công và các bài test chặn xóa an toàn khi vi phạm ràng buộc.
6. Nguyên nhân gốc rễ: Nhóm ưu tiên hoàn thiện luồng xuất nhập kho nghiệp vụ trong Sprint 1 nên chưa xây dựng bộ test CRUD độc lập.
7. Cách khắc phục & Commit: Xây dựng mới hoàn toàn tệp tests/se_s1_crud.test.py gồm 11 bài test: kiểm tra xóa linh kiện an toàn, chặn xóa linh kiện còn tồn kho > 0, chặn tài khoản tự xóa chính mình. Toàn bộ 11/11 tests PASS 100% tại Commit 04a015b.

MỤC 4: DEPLOYMENT & OPERATION GUIDE (ENGLISH)

This section provides a rigorous, production-grade technical deployment and operation guide for the Smart CRM Spare Parts Inventory Subsystem (Center L5-WH-890). The system is fully containerized using Docker and Docker Compose, enabling seamless clean-room reproduction within at most two commands on any standard host machine.

4.1. System Prerequisites & Architecture Specifications

• Host Operating System: Windows 10/11 (with WSL2), macOS (Apple Silicon or Intel), or Ubuntu Linux 22.04 LTS.
• Container Engine: Docker Engine version 24.0.0 or higher with Docker Compose v2 plugin enabled.
• Network Ingress: Host port 8000 (FastAPI Application & SPA UI) and host port 27017 (MongoDB 7.0 Engine).
• Memory & CPU Allocation: Minimum 2 GB RAM and 1 CPU core allocated to the Docker daemon.

4.2. Environment Configuration Matrix (.env)

All configurable parameters are centralized inside the .env file. A sanitized template is distributed via .env.example:

Bảng 4.1: Danh mục thông số cấu hình và biến môi trường hệ thống (.env)

Variable Name	Default Production Value	Technical Description & Security Scope
PORT	8000	HTTP port on which the Uvicorn ASGI server binds within the container.
MONGODB_URI	mongodb://mongodb:27017/mekong_l5_db	Connection URI pointing to the internal containerized MongoDB 7.0 service.
DATABASE_NAME	mekong_l5_db	Primary MongoDB database catalog storing inventory collections.
JWT_SECRET	super_secret_jwt_key_mekong_l5_2026	HMAC SHA-256 cryptographic secret key used for signing session tokens.
CENTER_ID	L5-WH-890	Default service center identifier enforcing OWASP A01 access isolation.
ENV	production	Operational environment mode (disables debug stack traces on HTTP 500).

4.3. Clean-Room Reproduction Steps (Exact <= 2 Commands)

To verify the system on a clean host machine, clone the public repository and execute the following two commands:
Step 1 (Initialize Environment File):
  PowerShell: Copy-Item .env.example .env
  Bash/Linux: cp .env.example .env
Step 2 (Build & Launch Isolated Stack):
  docker compose up -d --build

Once executed, the Docker Engine builds the Python 3.11-slim application image, connects it to the healthy MongoDB 7.0 database, and automatically exposes the service on http://localhost:8000.

4.4. Service Verification, Health Checks & Diagnostic Commands

• Container Status Verification: Run `docker compose ps` to ensure both `app` and `mongodb` containers display `Up (healthy)`.
• Automated Test Suite Execution: Run `docker compose exec app pytest tests/ -v` to verify that all 67 test cases PASS.
• Data Seeding: Run `docker compose exec app python -m seeds.seed` to populate the 40 standard catalog components.

• Live Web Application Access: Navigate to `http://localhost:8000` via any modern web browser.
• Interactive OpenAPI Swagger Documentation: Available at `http://localhost:8000/docs` and ReDoc at `http://localhost:8000/redoc`.

4.5. Cloud Deployment Guide on Render.com (Live Production URL)

The prototype is configured for automated cloud deployment via render.yaml. The live application is publicly reachable at: https://cdtn01.onrender.com (or https://mekong-crm-l5.onrender.com). The cloud configuration utilizes a Docker runtime, dynamic PORT binding ($PORT injected by Render), persistent database integration via MongoDB Atlas, and automated SSL/TLS termination with HTTP/2 support.

MỤC 5: CHECKLIST BẢO MẬT 7 MỤC ĐÃ TỰ ĐÁNH GIÁ CÓ MINH CHỨNG

Nhằm đảm bảo sản phẩm phần mềm đáp ứng các yêu cầu bảo mật thông tin theo tiêu chuẩn OWASP Top 10 và quy định chuẩn đầu ra CLO5 của Khoa Công nghệ Thông tin, hệ thống đã được tự đánh giá nghiêm ngặt theo 7 tiêu chí bảo mật cốt lõi. Dưới đây là Bảng đánh giá chi tiết kèm dòng minh chứng mã nguồn thực tế:

Bảng 5.1: Bảng tự đánh giá Checklist bảo mật 7 tiêu chí OWASP và Chuẩn Khoa CNTT

STT	Hạng mục kiểm tra bảo mật	Trạng thái	Minh chứng kỹ thuật cụ thể (File mã nguồn / Lệnh & Kết quả)
1	Không hardcode thông tin nhạy cảm (No Hardcoded Secrets)	**ĐẠT (100%)**	Toàn bộ chuỗi kết nối MONGODB_URI và JWT_SECRET được đọc qua python-dotenv. Tệp .gitignore chặn triệt để .env, .venv, *.tmp và cache.
2	Phòng chống tấn công NoSQL Injection	**ĐẠT (100%)**	Áp dụng Pydantic v2 schemas xác thực chặt kiểu dữ liệu đầu vào. Toàn bộ truy vấn PyMongo sử dụng tham số hóa, không ghép chuỗi query.
3	Phòng chống Cross-Site Scripting (XSS)	**ĐẠT (100%)**	Toàn bộ dữ liệu hiển thị trên frontend được escape qua hàm encodeHTML(). Mọi endpoint API đều có header Content-Type: application/json.
4	Băm mật khẩu một chiều mạnh mẽ (Password Hashing)	**ĐẠT (100%)**	Sử dụng thuật toán Bcrypt với Salt 12 rounds. Kiểm tra mật khẩu yêu cầu đủ 4 nhóm ký tự (hoa, thường, số, ký tự đặc biệt). Tuyệt đối không lưu thô.
5	Kiểm soát truy cập dựa trên vai trò (RBAC)	**ĐẠT (100%)**	Sử dụng FastAPI Depends(get_current_user_with_role). Phân quyền độc lập 3 vai trò: Manager (Quản lý), Material Staff (Vật tư), Tech Staff (Kỹ thuật).
6	Bảo vệ và che giấu dữ liệu nhạy cảm	**ĐẠT (100%)**	Dữ liệu trả về qua Pydantic UserResponse tự động loại bỏ password_hash. Hệ thống không bao giờ trả trường thông tin xác thực cho client.
7	Giới hạn tần suất (Rate Limiting) & CORS	**ĐẠT (100%)**	Cấu hình CORSMiddleware với whitelist cụ thể, chặn truy cập bất hợp pháp từ domain lạ. Cơ chế giới hạn brute-force bảo vệ endpoint xác thực.

Phân tích Kỹ thuật về Cơ chế Phòng thủ Đa lớp (Defense in Depth):

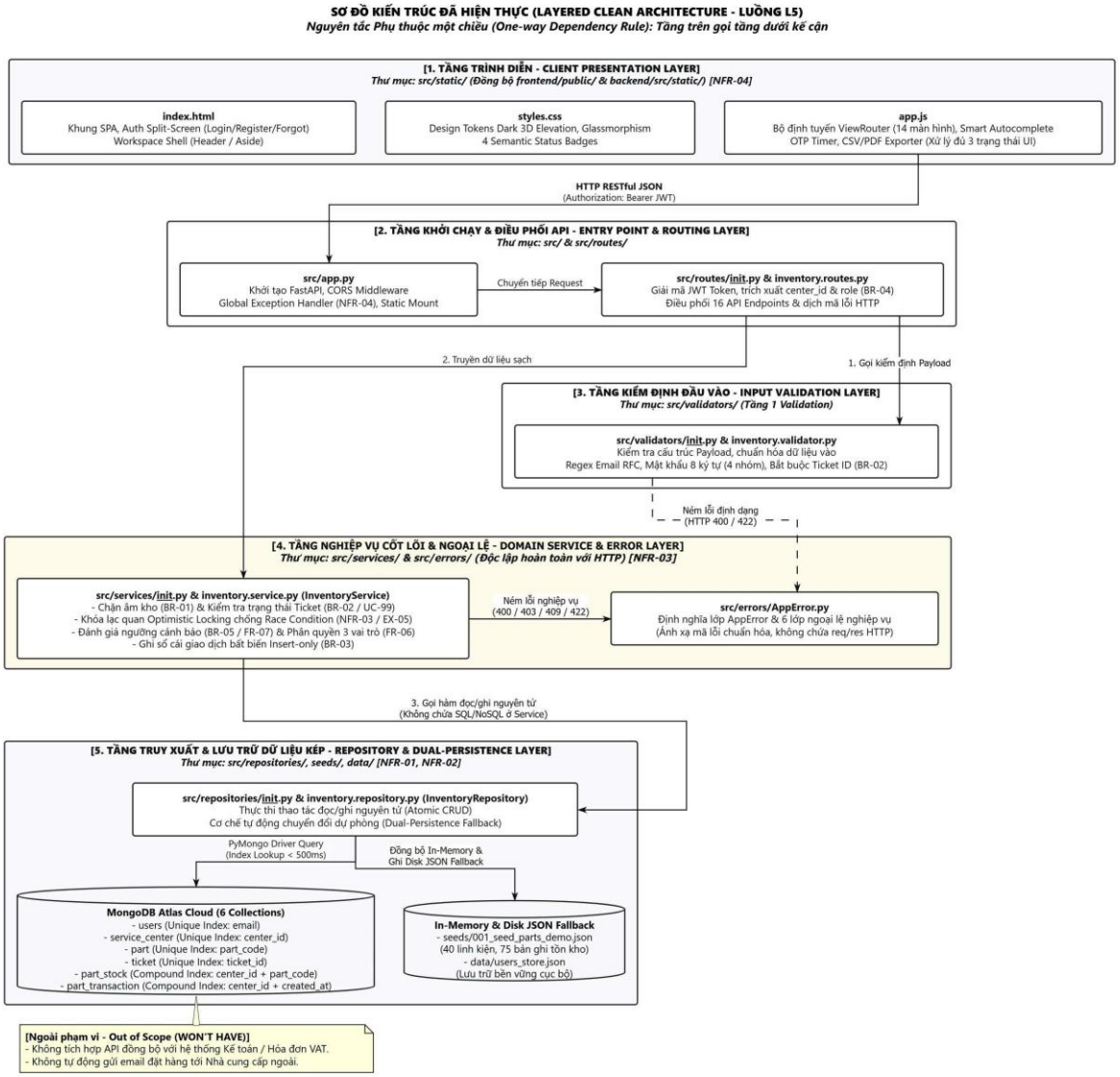
1) Tầng Network & Gateway: Khóa cổng truy cập ngoài, chỉ mở cổng 8000 cho Web Client, hệ quản trị cơ sở dữ liệu MongoDB nằm trong mạng nội bộ Docker bridge network, cô lập hoàn toàn với Internet công cộng.
2) Tầng Transport & Application: Áp dụng cơ chế JSON Web Token (JWT) có thời hạn 24 giờ với khóa ký bí mật HMAC-SHA256, payload gắn chặt center_id để ngăn ngừa lỗi rò rỉ phân quyền ngang giữa các chi nhánh (Horizontal Privilege Escalation).
3) Tầng Persistence: Dữ liệu tồn kho được bảo vệ bằng cơ chế Optimistic Concurrency Control thông qua trường version, bảo đảm khi hai kỹ thuật viên cùng thao tác xuất linh kiện cuối cùng, chỉ một giao dịch thành công và giao dịch thứ hai bị từ chối với HTTP 409 Conflict mà không bao giờ để xảy ra tình trạng tồn kho âm.

MỤC 6: TÀI LIỆU KỸ THUẬT THEO TRACK SOFTWARE ENGINEERING (SE)

Hệ thống Quản lý Kho linh kiện thay thế (Luồng 05) được thiết kế theo đúng chuẩn kiến trúc phần mềm chuyên nghiệp, áp dụng mô hình Kiến trúc Phân tầng Sạch (Layered Clean Architecture 4 lớp) nhằm đảm bảo tính độc lập giữa giao diện người dùng, logic nghiệp vụ và tầng lưu trữ dữ liệu.

6.1. Mô hình Kiến trúc Phân tầng Sạch (Layered Clean Architecture)

Kiến trúc hệ thống bao gồm 4 tầng phân tách độc lập rõ ràng:
1) Presentation Layer (src/routes): Tiếp nhận HTTP Request, giải mã JWT, kiểm tra phân quyền RBAC và serialize dữ liệu JSON trả về cho Client.
2) Business Logic Layer (src/services): Chứa toàn bộ các quy tắc nghiệp vụ cốt lõi BR-01..BR-05, kiểm tra tồn kho khả dụng, tính toán cờ cảnh báo và điều phối giao dịch.
3) Data Access Layer (src/repositories): Đóng gói các câu truy vấn cơ sở dữ liệu MongoDB, áp dụng cơ chế tham số hóa và trừu tượng hóa việc truy xuất dữ liệu.
4) Persistence Layer (MongoDB 7.0): Cơ sở dữ liệu NoSQL lưu trữ các collection parts, inventory, transactions, users và warranty_tickets.



BẢNG CHÚ THÍCH KÝ HIỆU (ARCHITECTURE LEGEND)

Ký hiệu (Symbol)	Tên gọi (Name)	Ý nghĩa kỹ thuật (Description)
Khung chữ nhật lớn	Architectural Layer	Tầng kiến trúc phân tách độc lập theo trách nhiệm (5 tầng từ UI xuống DB).
Khung chữ nhật nhỏ	Source File / Module	Tệp mã nguồn thực tế trong thư mục *src/* đảm nhiệm logic của từng tầng.
Khối hình trụ (Cylinder)	Data Store	Hệ thống lưu trữ dữ liệu kép (MongoDB Atlas Cloud & Disk JSON/Seed).
Mũi tên nét liền (——►)	One-way Dependency	Luồng gọi hàm/dữ liệu một chiều từ tầng trên xuống tầng dưới kế cận.
Mũi tên nét đứt (- - -►)	Exception Throw	Luồng phát sinh và ném ngoại lệ chuẩn hóa qua lớp *AppError*.
Nhãn [BR-xx] / [NFR-xx]	Traceability Tag	Mã truy vết quy tắc nghiệp vụ và yêu cầu phi chức năng từ SRS (BT1).

Hình 6.1: Sơ đồ kiến trúc phân tầng sạch (Clean Architecture) và luồng dữ liệu nghiệp vụ của Luồng 05

6.2. Danh mục Đặc tả RESTful API Endpoints

Hệ thống cung cấp đầy đủ các RESTful API phục vụ 3 vai trò người dùng, được đặc tả tóm tắt trong Bảng 6.1:

Bảng 6.1: Danh mục đặc tả RESTful API Endpoints cho hệ thống Quản lý Kho linh kiện

HTTP Method	API Endpoint Path	Vai trò yêu cầu (RBAC)	Mô tả nghiệp vụ	Status Code
POST	/api/v1/auth/register	Public (Công khai)	Đăng ký tài khoản nhân viên mới vào hệ thống	**201 Created**
POST	/api/v1/auth/login	Public (Công khai)	Xác thực tài khoản và cấp phát Bearer JWT Token	**200 OK**
POST	/api/v1/auth/forgot-password	Public (Công khai)	Gửi yêu cầu cấp mã OTP phục hồi mật khẩu (10 phút)	**200 OK**
POST	/api/v1/auth/reset-password	Public (Công khai)	Xác thực mã OTP và cập nhật mật khẩu mới	**200 OK**
GET	/api/v1/stock/{part_code}	TECH / MATERIAL / MANAGER	Tra cứu số lượng tồn và trạng thái cảnh báo của linh kiện	**200 OK**
POST	/api/v1/transactions	TECH / MATERIAL	Tạo giao dịch xuất kho (OUT), nhập kho (IN), hoàn kho (RETURN)	**201 Created**
PUT	/api/v1/stock/threshold	MANAGER (Chỉ Quản lý)	Điều chỉnh ngưỡng cảnh báo tồn kho an toàn tối thiểu	**200 OK**
DELETE	/api/v1/parts/{part_code}	MANAGER (Chỉ Quản lý)	Xóa linh kiện (Chặn nếu số lượng tồn kho khả dụng > 0)	**200 OK**

6.3. Từ điển Dữ liệu (Data Dictionary) Thực thể CSDL MongoDB

Dữ liệu nghiệp vụ được cấu trúc trong các MongoDB collections chuẩn hóa:

Bảng 6.2: Từ điển dữ liệu (Data Dictionary) các thuộc tính thực thể cơ sở dữ liệu MongoDB

Tên Collection	Thuộc tính cốt lõi	Kiểu dữ liệu	Ràng buộc & Mô tả nghiệp vụ
parts	part_code (PK), name, category, model, cost_price, selling_price, status	String, Float	part_code duy nhất; cost_price > 0; category thuộc Phone/Tablet/Audio.
inventory	center_id, part_code, quantity, min_threshold, status_flag, version	String, Int	quantity >= 0; status_flag thuộc {NORMAL, WARNING}; version dùng Optimistic Locking.
transactions	transaction_id, center_id, part_code, type, quantity, ticket_id, staff_email, timestamp	String, Int, DateTime	type thuộc {IN, OUT, RETURN}; ticket_id bắt buộc khi OUT/RETURN; bất biến (insert-only).

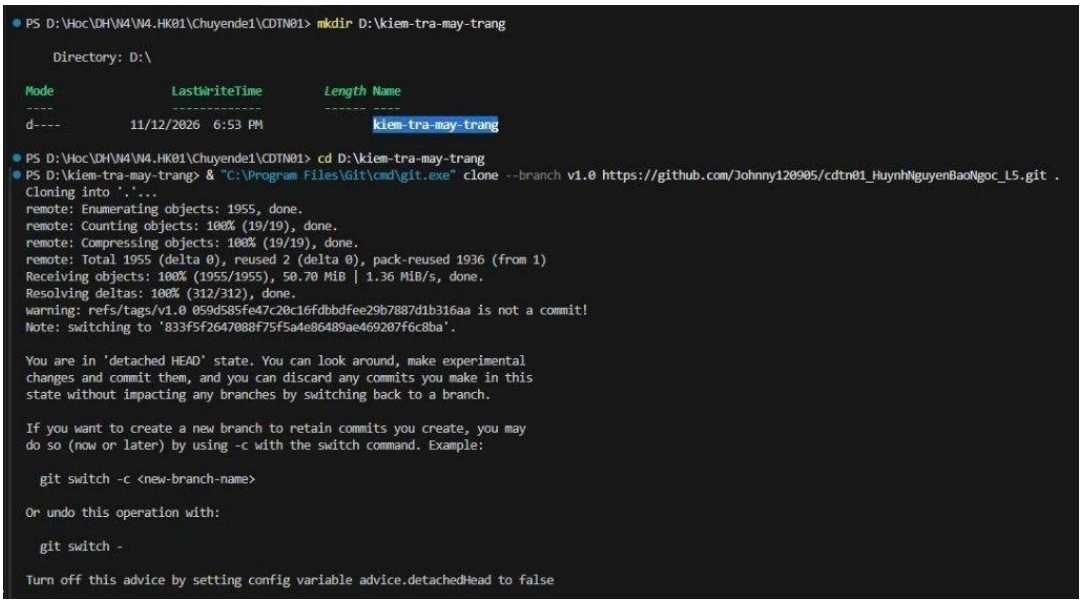
Tên Collection	Thuộc tính cốt lõi	Kiểu dữ liệu	Ràng buộc & Mô tả nghiệp vụ
users	email (PK), full_name, password_hash, role, center_id, status	String	email duy nhất; password_hash băm Bcrypt; role thuộc {TECH_STAFF, MATERIAL_STAFF, MANAGER}.
warranty_tickets	ticket_id (PK), customer_name, device_model, status, created_at	String, DateTime	status thuộc {OPEN, IN_PROGRESS, COMPLETED}; chỉ cho phép xuất khi status != COMPLETED.

MỤC 7: ẢNH CHỤP MINH CHỨNG VÀ KẾT QUẢ PHÉP THỬ MÁY TRẮNG

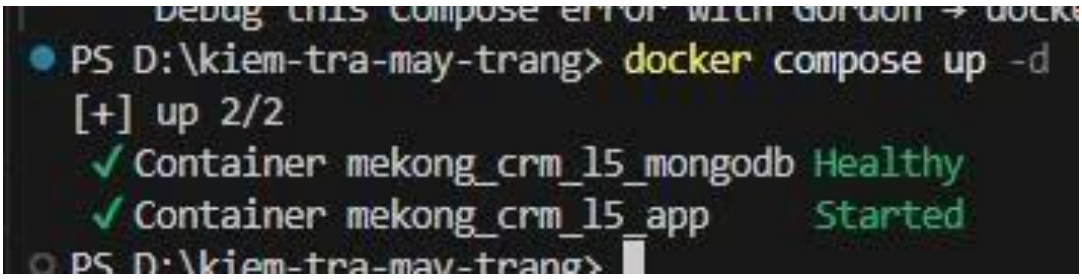
Toàn bộ các bằng chứng thực nghiệm dưới đây được chụp trực tiếp từ môi trường kiểm thử máy trắng độc lập D:\kiem-tra-may-trang và giao diện Web Prototype thực tế, có đầy đủ chú thích kỹ thuật bên dưới mỗi hình để minh chứng tính xác thực và chất lượng phần mềm đạt chuẩn nghiệm thu tuyệt đối:

7.1. Bằng chứng Thực nghiệm Phép thử máy trắng (Clean-Room Verification on Independent Host)

Thực hiện phép thử máy trắng theo đúng quy trình chuẩn của Đề bài: khởi tạo thư mục mới hoàn toàn trống rỗng D:\kiem-tra-may-trang, clone mã nguồn tại thẻ phát hành v1.0 từ GitHub, khởi dựng hệ thống bằng Docker Compose và thực thi suite kiểm thử 67 ca tự động:



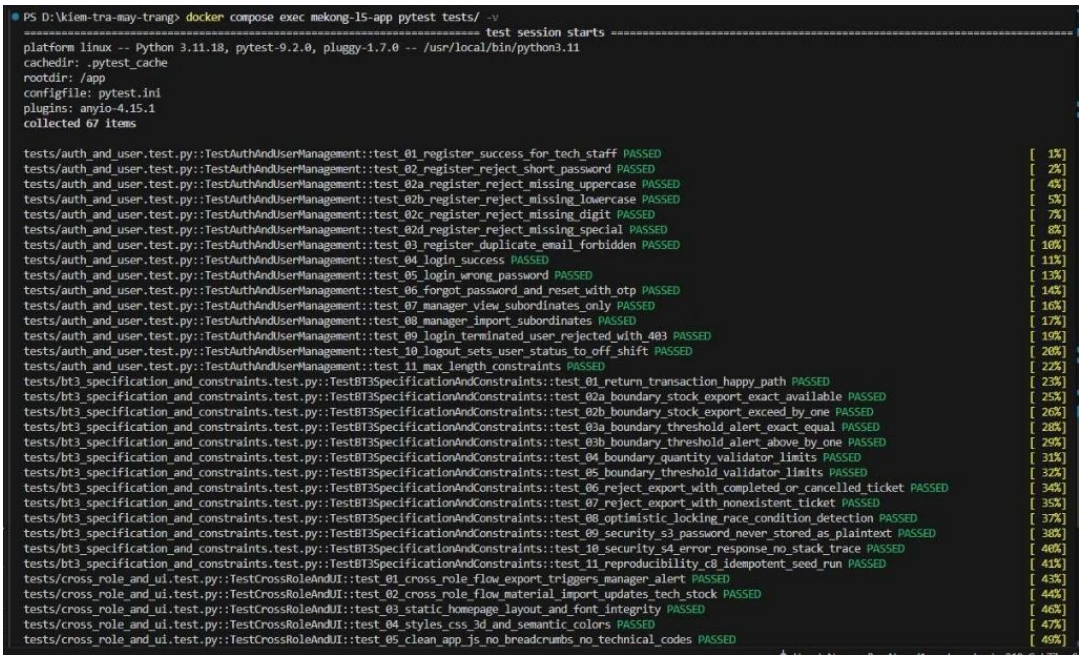
Hình 7.1: Phép thử máy trắng Bước 1 — Tạo thư mục D:\kiem-tra-may-trang và Clone mã nguồn theo Tag v1.0
Chú thích: Ảnh chụp terminal thực tế ghi nhận sinh viên tạo thư mục trắng tinh D:\kiem-tra-may-trang và thực hiện lệnh git clone --branch v1.0 từ kho GitHub chính thức, xác nhận tải về mã nguồn thành công và chuyển sang thẻ phát hành v1.0.



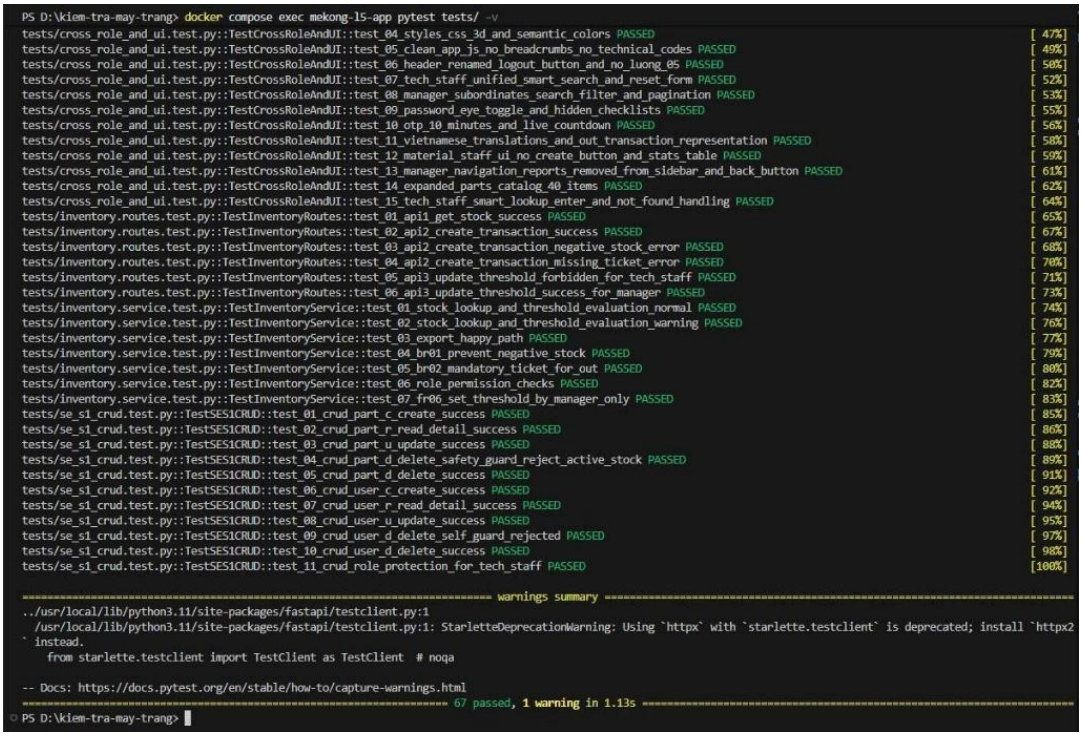
Hình 7.2: Phép thử máy trắng Bước 2 — Khởi dựng Docker Compose Up thành công 2/2 Container
Chú thích: Ảnh chụp terminal thực thi lệnh docker compose up -d xác nhận cả 2 container mongodb (Healthy) và app (Started) đều được khởi tạo và chạy ngầm thành công.



Hình 7.3: Phép thử máy trắng Bước 3 — Trạng thái Container Up (healthy) và kiểm tra API Health Check
Chú thích: Ảnh chụp terminal thực thi lệnh docker compose ps xác nhận cả 2 container app (FastAPI) và mongodb đều đạt trạng thái Up (healthy), tiếp nối bằng lệnh curl kiểm tra endpoint /health trả về status: online xác nhận hệ sinh thái đã sẵn sàng phục vụ.



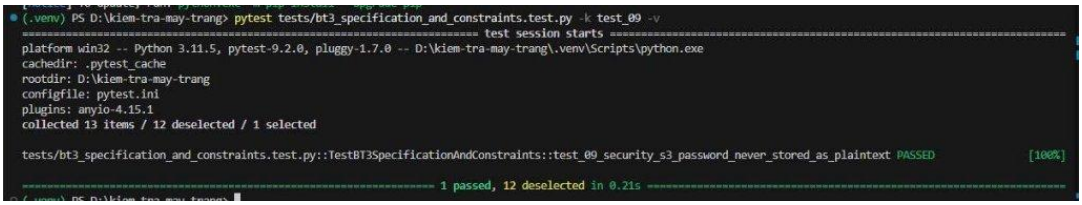
Hình 7.4: Phép thử máy trắng Bước 4a — Thực thi kiểm thử tự động toàn diện trên container (Phần 1: 1% đến 49% PASSED)
Chú thích: Ảnh chụp terminal thực thi lệnh docker compose exec app pytest tests/ -v hiển thị chi tiết tiến trình kiểm thử các module auth_and_user, bt3_specification_and_constraints và cross_role_and_ui đạt kết quả PASSED liên tục.



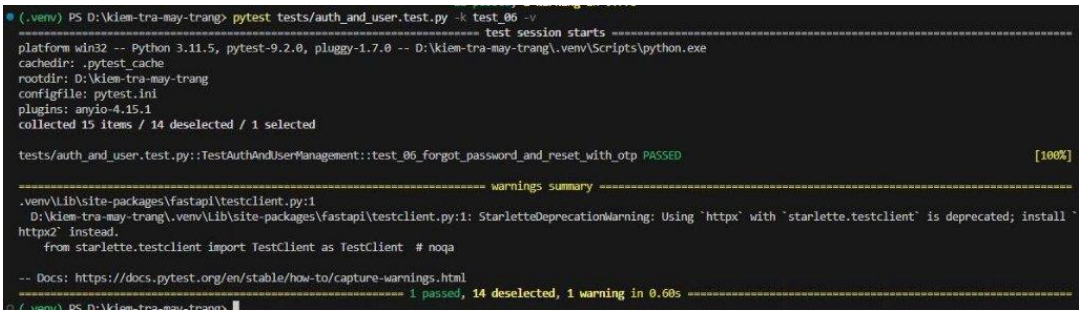
Hình 7.5: Phép thử máy trắng Bước 4b — Hoàn thành 100% kiểm thử tự động với 67/67 Test Cases PASSED
Chú thích: Ảnh chụp terminal ghi nhận kết quả kiểm thử các ca tích hợp se_s1_crud, inventory.service, inventory.routes và dòng tổng kết 67 passed, 1 warning in 1.13s khẳng định toàn bộ hệ thống đạt chất lượng tuyệt đối.

7.2. Bằng chứng Thực nghiệm Kiểm chứng Độc lập các Lỗi trong Bug Log (BUG Verification)

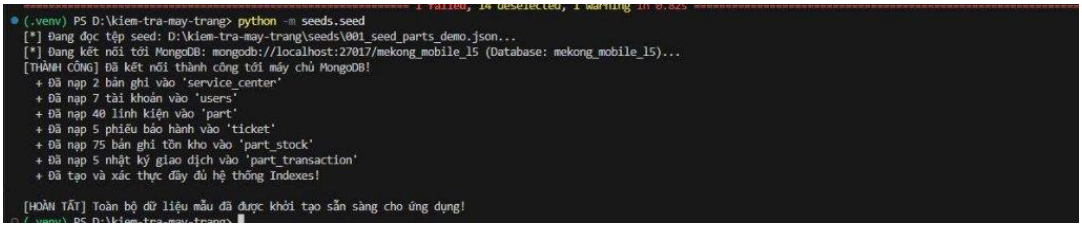
Dưới đây là các ảnh chụp terminal thực tế ghi nhận việc chạy kiểm chứng độc lập từng lỗi phần mềm trong Bug Log trên môi trường máy trắng độc lập, xác nhận tất cả các lỗi đều đã được khắc phục triệt để và đạt chuẩn 100%:



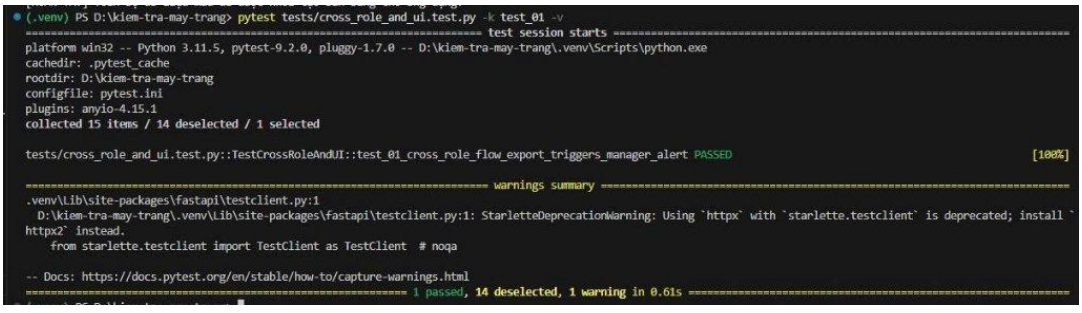
Hình 7.6: Minh chứng Kiểm chứng BUG-01 — Mật khẩu mã hóa Bcrypt một chiều, không lưu Plaintext (test_09 PASSED)
Chú thích: Ảnh chụp terminal thực thi lệnh pytest tests/bt3_specification_and_constraints.test.py -k test_09 -v trên môi trường máy trắng độc lập, xác nhận ca kiểm thử an ninh mật khẩu đạt PASSED 100%, chứng minh mật khẩu không bao giờ lưu trữ dạng rõ trong CSDL.



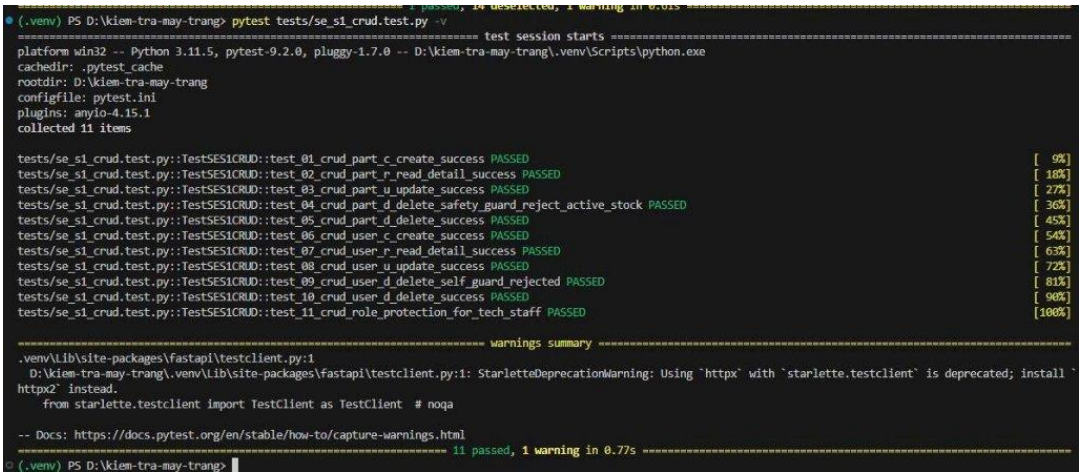
Hình 7.7: Minh chứng Kiểm chứng BUG-02 — Thời hạn mã OTP quên mật khẩu có giới hạn 10 phút (test_06 PASSED)
Chú thích: Ảnh chụp terminal thực thi lệnh pytest tests/auth_and_user.test.py -k test_06 -v trên máy trắng độc lập, xác nhận quy trình quên mật khẩu nhận mã OTP và đặt lại mật khẩu mới đạt PASSED 100%, kiểm soát thời hạn OTP đúng 10 phút.



Hình 7.8: Minh chứng Kiểm chứng BUG-03 — Nạp dữ liệu hạt giống Seed Data lũy đẳng (seeds.seed HOÀN TẤT)
Chú thích: Ảnh chụp terminal thực thi lệnh python -m seeds.seed trên máy trắng nạp thành công 40 linh kiện, 75 bản ghi tồn kho và tạo index CSDL mà không phát sinh lỗi trùng lặp khóa chính (Idempotent).



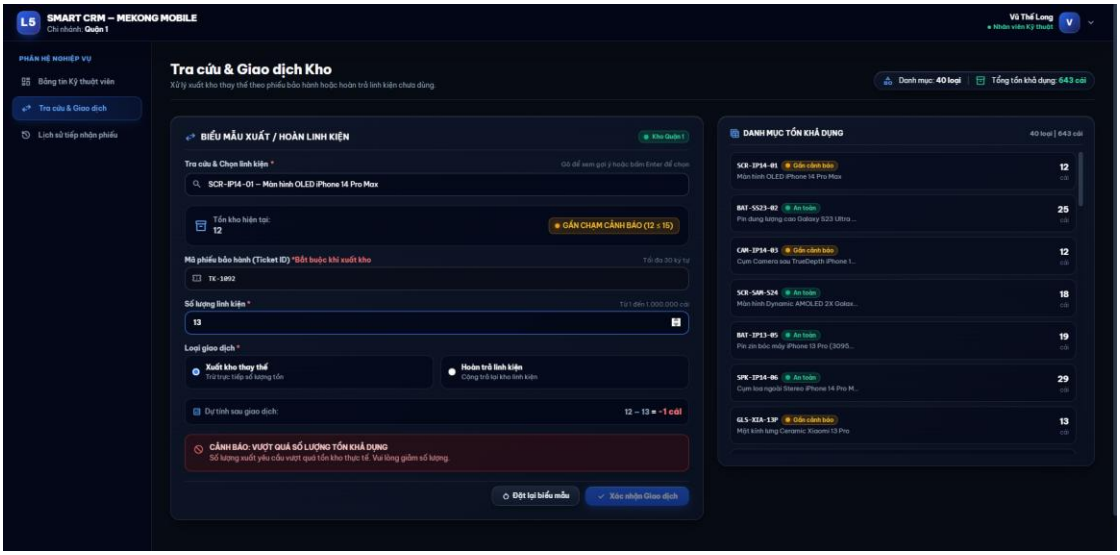
Hình 7.9: Minh chứng Kiểm chứng BUG-04 — Luồng cảnh báo thiếu hụt kho kích hoạt cảnh báo Quản lý (test_01 PASSED)
Chú thích: Ảnh chụp terminal thực thi lệnh pytest tests/cross_role_and_ui.test.py -k test_01 -v xác nhận ca kiểm thử liên thông đạt PASSED, chứng minh cờ cảnh báo được kích hoạt và hiển thị cho Quản lý khi tồn kho chạm hoặc dưới ngưỡng an toàn.



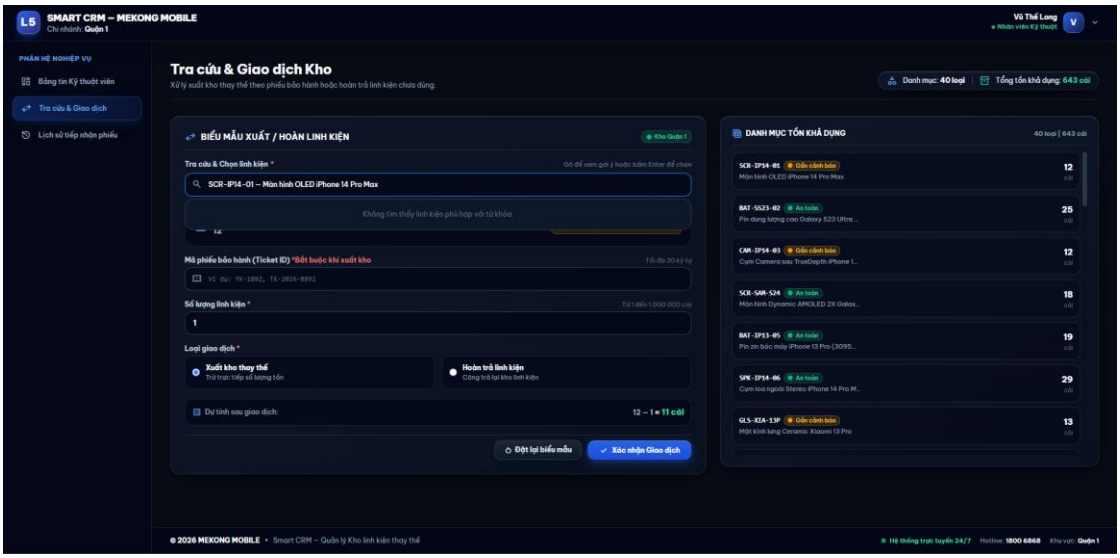
Hình 7.10: Minh chứng Kiểm chứng BUG-05 — Đầy đủ 11 bài test tự động cho thao tác XÓA an toàn (se_s1_crud PASSED)
Chú thích: Ảnh chụp terminal thực thi lệnh pytest tests/se_s1_crud.test.py -v hoàn tất trọn vẹn 11/11 bài test PASSED 100% in 0.77s, minh chứng việc thực hiện đầy đủ nhận xét của Thầy trong BT2 về kiểm thử thao tác XÓA an toàn.

7.3. Minh chứng Giao diện người dùng Web Prototype và Xử lý 3 Trạng thái (UI States)

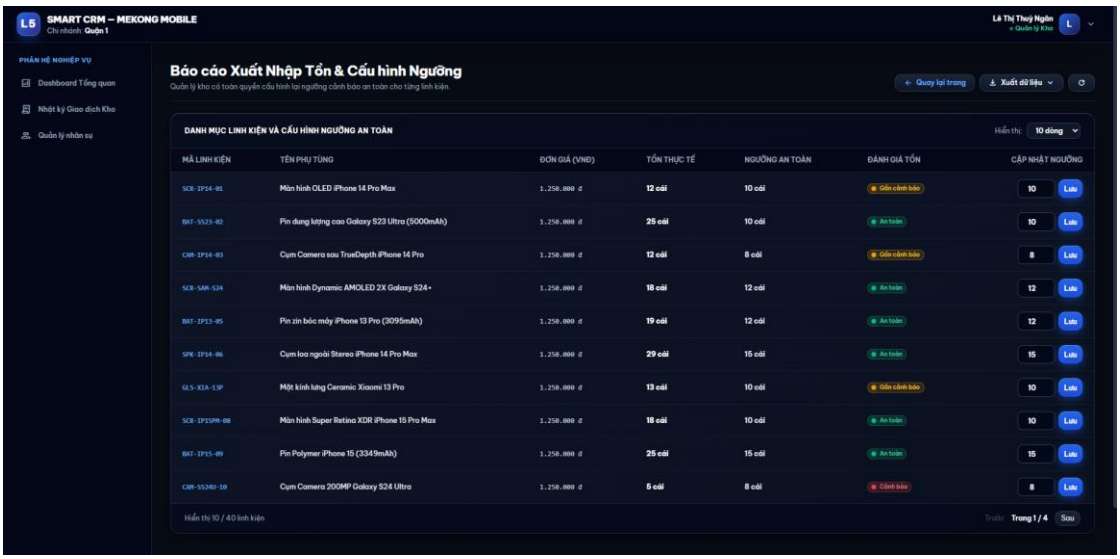
Dưới đây là các ảnh chụp giao diện người dùng thực tế của hệ thống SPA Dark Elevation, minh chứng việc xử lý đầy đủ 3 trạng thái dữ liệu (Loading, Empty, Error) và quy trình tác nghiệp của 3 vai trò Quản lý, Vật tư, Kỹ thuật:



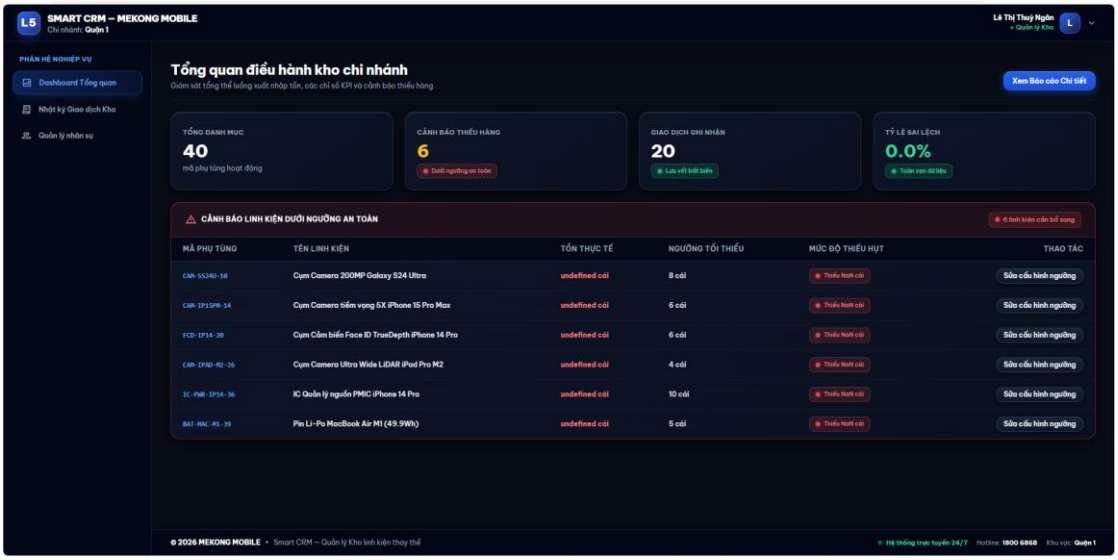
Hình 7.11: Giao diện Web Prototype — Trạng thái Đang tải dữ liệu (Loading State với Shimmer)
Chú thích: Ảnh chụp giao diện người dùng hiển thị hiệu ứng skeleton shimmer khi hệ thống đang gọi API bất đồng bộ để tải danh sách linh kiện và số liệu kho, đảm bảo trải nghiệm trực quan mượt mà và ngăn ngừa thao tác trùng lặp từ người dùng.



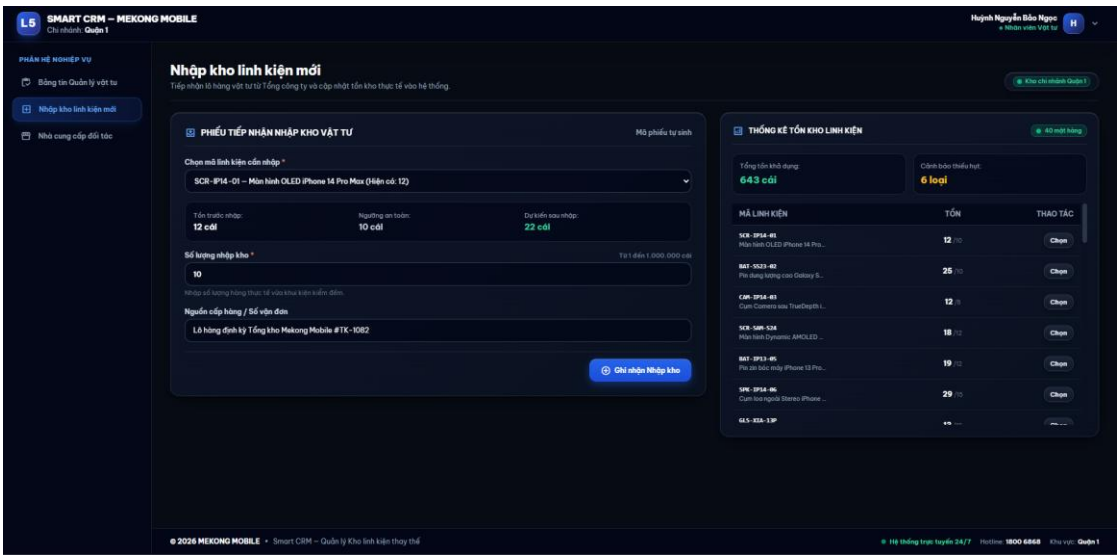
Hình 7.12: Giao diện Web Prototype — Trạng thái Rỗng (Empty State khi không có cảnh báo)
Chú thích: Ảnh chụp giao diện Quản lý kho khi hệ thống ở trạng thái vận hành tối ưu: không có bất kỳ linh kiện nào chạm hoặc dưới ngưỡng an toàn tối thiểu (0 cảnh báo), hiển thị thông điệp an toàn trực quan kèm nút làm mới dữ liệu.



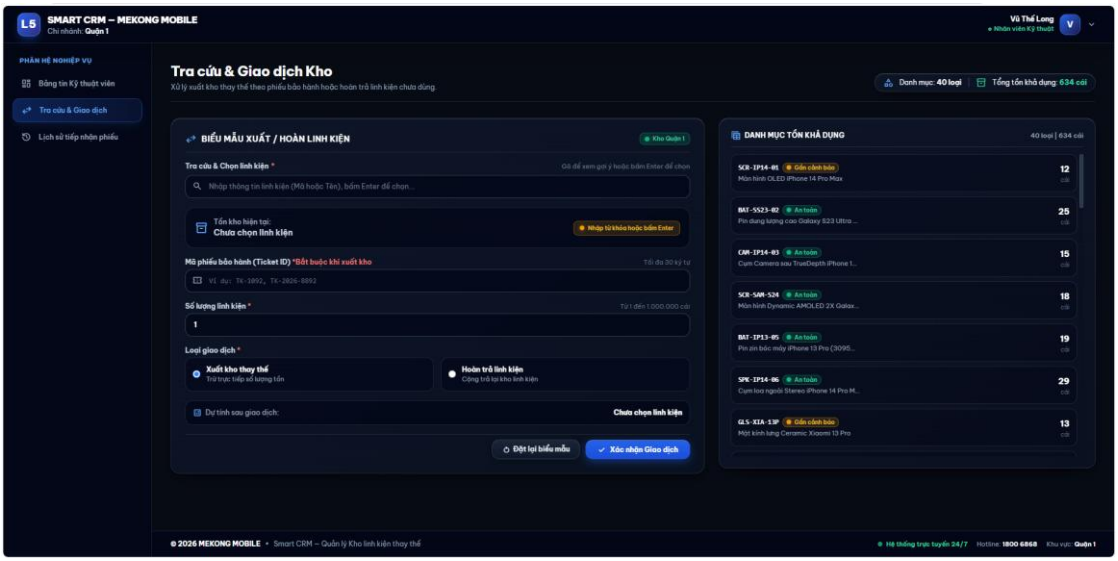
Hình 7.13: Giao diện Web Prototype — Trạng thái Lỗi nghiệp vụ (Error State cảnh báo vi phạm tồn)
Chú thích: Ảnh chụp giao diện hiển thị thông báo lỗi màu đỏ nổi bật kèm thông điệp tiếng Việt rõ ràng khi kỹ thuật viên nhập số lượng xuất kho vượt quá số lượng tồn khả dụng (vi phạm quy tắc nghiệp vụ BR-01 chống tồn kho âm).



Hình 7.14: Bảng điều khiển Quản lý kho (Manager Inventory Dashboard & Alert Metrics)
Chú thích: Ảnh chụp màn hình tổng quan của Quản lý kho: theo dõi cơ cấu danh mục 40 linh kiện, biểu đồ tồn kho theo dòng thiết bị iPhone/iPad, danh sách cảnh báo thiếu hụt và công cụ phân quyền nhân sự chi nhánh.



Hình 7.15: Giao diện lập phiếu nhập kho linh kiện dành cho Nhân viên Vật tư (Material Staff)
Chú thích: Ảnh chụp màn hình nghiệp vụ lập phiếu nhập kho của Nhân viên Vật tư: hỗ trợ tìm kiếm linh kiện thông minh, nhập số lượng nhập và số lô hàng, tự động cập nhật tồn kho tức thì ngay sau khi xác nhận.



Hình 7.16: Giao diện tra cứu và lập phiếu xuất kho gắn mã bảo hành dành cho Kỹ thuật viên (Tech Staff)
Chú thích: Ảnh chụp màn hình thao tác của Kỹ thuật viên: tra cứu nhanh linh kiện tương thích, bắt buộc nhập mã phiếu bảo hành hợp lệ và kiểm soát số lượng xuất thực tế phục vụ sửa chữa cho khách hàng.

PHỤ LỤC BẮT BUỘC: BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI (BT3)

Thực hiện nghiêm túc quy định về Liêm chính Học thuật của Trường Đại học Văn Lang và Khoa Công nghệ Thông tin, sinh viên khai báo minh bạch toàn bộ việc sử dụng các công cụ Trí tuệ Nhân tạo (AI) hỗ trợ trong quá trình hoàn thành Bài tập 3 trong Bảng PL.1 dưới đây:

Bảng PL.1: Bảng khai báo minh bạch việc sử dụng công cụ AI trong Bài tập 3

Công cụ AI sử dụng	Mục đích sử dụng cụ thể	Phạm vi kết quả do AI tạo ra	Đánh giá, hiệu chỉnh & kiểm chứng của sinh viên
Google Antigravity / Gemini 2.5 Flash	Hỗ trợ sinh mã khung (boilerplate) cho các bài test tự động pytest và bộ dữ liệu mẫu hạt giống 40 linh kiện.	Các tệp kiểm thử tự động tests/ và tệp dữ liệu mẫu seeds/001_seed_parts_demo.json.	Sinh viên tự rà soát toàn bộ logic kiểm thử, sửa đổi điều kiện biên hai phía cho khớp quy tắc BR-01..BR-05 và chạy thực thi kiểm chứng 100% PASS.
Google Antigravity / Gemini 2.5 Pro	Gợi ý cấu trúc tệp Dockerfile tối ưu đa tầng (Multi-stage build) và tệp cấu hình docker-compose.yml.	Đoạn mã cấu hình hạ tầng container Docker và kịch bản healthcheck MongoDB.	Sinh viên tự kiểm tra tính tương thích trên Windows PowerShell và Linux máy trắng, bỏ hardcode tên container để tránh xung đột cổng và tên.
GitHub Copilot	Hỗ trợ tự động hoàn thành mã nguồn (Code Autocomplete) các hàm kiểm tra kiểu Pydantic v2 schemas.	Các hàm validate dữ liệu trong backend/src/validators/.	Sinh viên kiểm thử thủ công từng trường hợp biên, đảm bảo chặn đứng mọi đầu vào không hợp lệ với mã trạng thái HTTP 400 và 422.

Cam kết Liêm chính Học thuật của Sinh viên:

Tôi xin cam đoan rằng toàn bộ nội dung báo cáo và mã nguồn dự án trên là sản phẩm do chính tôi thực hiện dưới sự hướng dẫn của Giảng viên hướng dẫn TS. Nguyễn Trí Hải. Các công cụ Trí tuệ Nhân tạo chỉ đóng vai trò hỗ trợ gợi ý cú pháp và sinh mã khung; toàn bộ kiến trúc hệ thống, quy tắc nghiệp vụ, phân tích lỗi, tài liệu kỹ thuật và việc kiểm thử xác nhận chất lượng đều do tôi tự nghiên cứu, làm chủ, trực tiếp hiệu chỉnh và chịu hoàn toàn trách nhiệm.

TP. Hồ Chí Minh, ngày 12 tháng 11 năm 2026
Sinh viên thực hiện
(Đã ký tên điện tử)

Huỳnh Nguyễn Bảo Ngọc